

Реализация паттерна Builder+Decorator

1. **Builder**: порождающий паттерн проектирования для пошагового конструирования сложных объектов; позволяет создавать разные варианты объектов (с разной функциональностью).
2. **Builder**: обычно логика Builder выносится в отдельный класс. Обычно Builder-класс – внутренний класс основного класса с приватным конструктором, экземпляр Builder создается с помощью публичного статического метода (по аналогии Singleton, но является им).

```
public partial class Calc // основной класс
{
    public static Builder getBuilder()... // получить экземпляр Builder
    public class Builder... // внутренний публичный класс Builder
}
```

```
var builder = Calc.getBuilder(); // получить экземпляр Builder
```

3. **Builder**: включает методы, представляющие «строительные блоки», создающие и настраивающие новый объект – результат работы Builder. Обычно выполнение методов связываются в цепочки. Каждая цепочка создает объект с требуемыми характеристиками.

```
public class Builder
{
    ICalcResult result = new Result(); // результат Decorator
    public static Builder Create()... // создать Builder
    Builder() {} // приватный конструктор
    public Builder Sum(int y)... // строительный блок: Sum
    public Builder Sub(int y)... // строительный блок: Sub
    public Builder Mul(int y)... // строительный блок: Mul
    public Builder Div(int y)... // строительный блок: Div
    public Builder Mod(int y)... // строительный блок: Mod
    public ICalcResult Build()... // строительный блок: конечный
}
```

```
var builder = Calc.getBuilder(); // получить экземпляр Builder
```

```
ICalcResult calcresult = builder.Mul(2).Sub(5).Mul(2).Sum(3).Build(); // цепочка генерирующая объект
```

```
int r = calcresult.result(3); // (((3*2)-5)*2)+3 = 5
r = calcresult.result(-4); // (((-4*2)-5)*2)+3 = -23
r = calcresult.result(0); // (((0*2)-5)*2)+3 = -7
```

4. **Builder:** каждый «строительный блок» «накручивает» функциональность на предшествующую цепочку блоков с помощью Decorator.

```
public partial class Calc // основной класс
{
    public static Builder getBuilder()... // получить экземпляр Builder
    public class Builder
    {
        ICalcResult result = new Result(); // результат Decorator
        public static Builder Create()... // создать Builder
        Builder() {} // приватный конструктор
        public Builder Sum(int y)
        {
            this.result = new Sum(y, this.result); // Decorator
            return this;
        }
        public Builder Sub(int y)
        {
            this.result = new Sub(y, this.result); // Decorator
            return this;
        }
        public Builder Mul(int y)... // строительный блок: Mul
        public Builder Div(int y)... // строительный блок: Div
        public Builder Mod(int y)... // строительный блок: Mod
        public ICalcResult Build()... // строительный блок: конечный
    }
}
```

5. **Decorator:** структурный паттерн проектирования, позволяющий динамически (в runtime) наращивать функциональность для объекта.

```
public interface ICalcResult
{
    int result(int y);
}
internal class Decorator : ICalcResult
{
    public int y = 0;
    public Decorator(int y) { this.y = y; }
    public virtual int result(int y)
    {
        return this.y = y;
    }
}
internal class Result : Decorator
{
    public Result() : base(0) { }
    public override int result(int y) { return y; }
}
internal class Sum : Decorator
{
    ICalcResult d;
    public Sum(int y, ICalcResult d):base(y){this.d = d;}
    public override int result(int y)
    {
        return d.result(y) + this.y;
    }
}
internal class Sub ...
internal class Mul ...
internal class Div ...
internal class Mod ...
```

6. Вопросы:

- 1) как сделать так, чтобы Builder можно было переписать?
- 2) как сделать независимыми Builder и Decorator?
- 3) как сделать расширяемым Calc?

NUGET

7. **NUGET**: .NET менеджер пакетов, nuget.org, dotnet-командная строка

8. **NUGET**: .csproj – файл проекта, информация о пакете

```
<PropertyGroup>
  <TargetFramework>net8.0</TargetFramework>
  <ImplicitUsings>enable</ImplicitUsings>
  <Nullable>enable</Nullable>

  <!-- для NuGet -->
  <!-- автогенерация NuGet-пакета -->
  <PackageId>BSTU.DP001</PackageId>
  <Version>1.0.0</Version>
  <Authors>smw60</Authors>
  <Company>BSTU</Company>
  <Description> Design Patterns, Lab01 </Description>
  <PackageTags>.NET, C#, Design Patterns, Builder, Decorator </PackageTags>
  <RepositoryUrl>https://github.com/smw60/DesignPatterns/Lab01</RepositoryUrl>
  <RepositoryType>git</RepositoryType>
  <PackageLicenseExpression>MIT</PackageLicenseExpression>
  <PackageReadmeFile>README.md</PackageReadmeFile>
</PropertyGroup>

</Project>
```

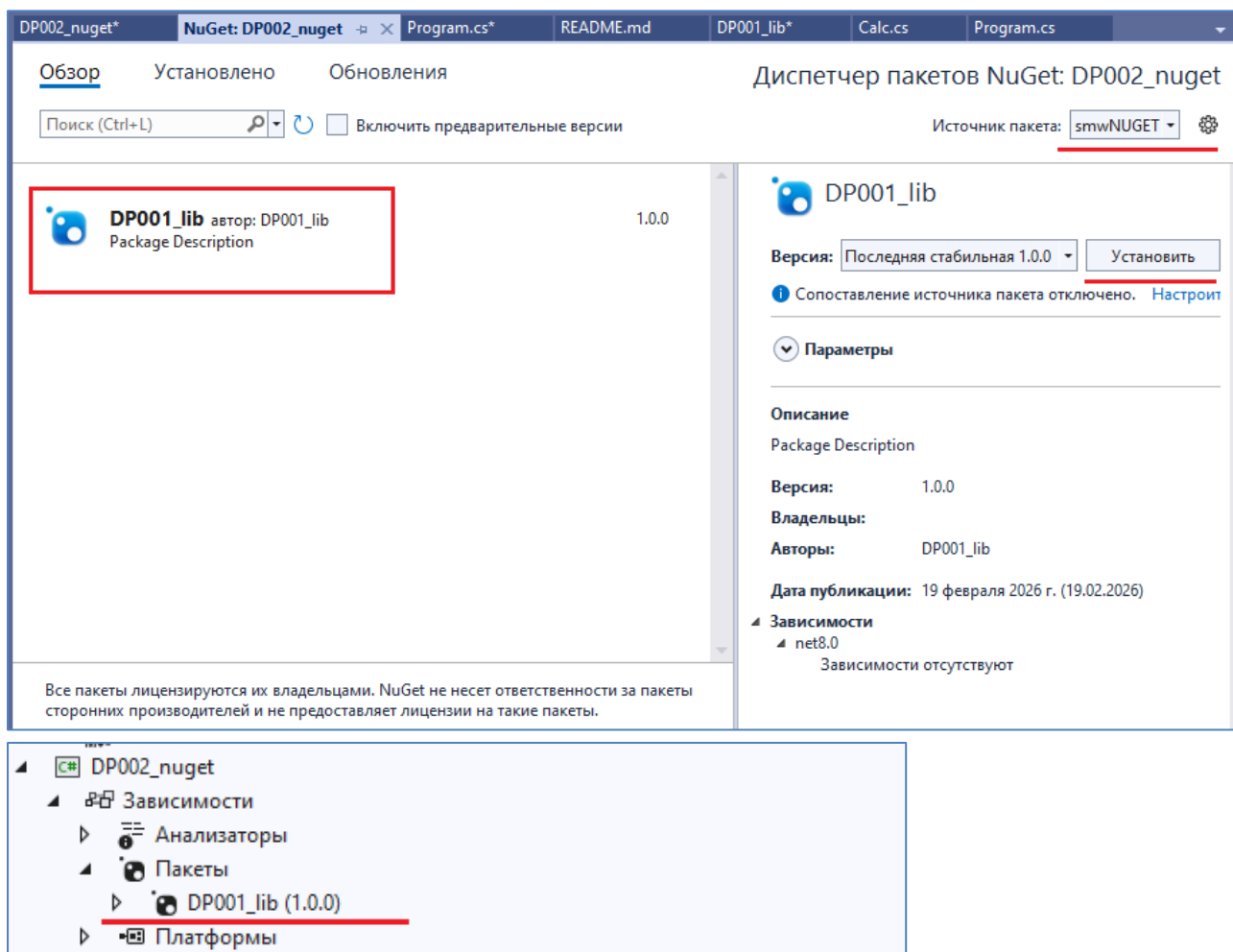
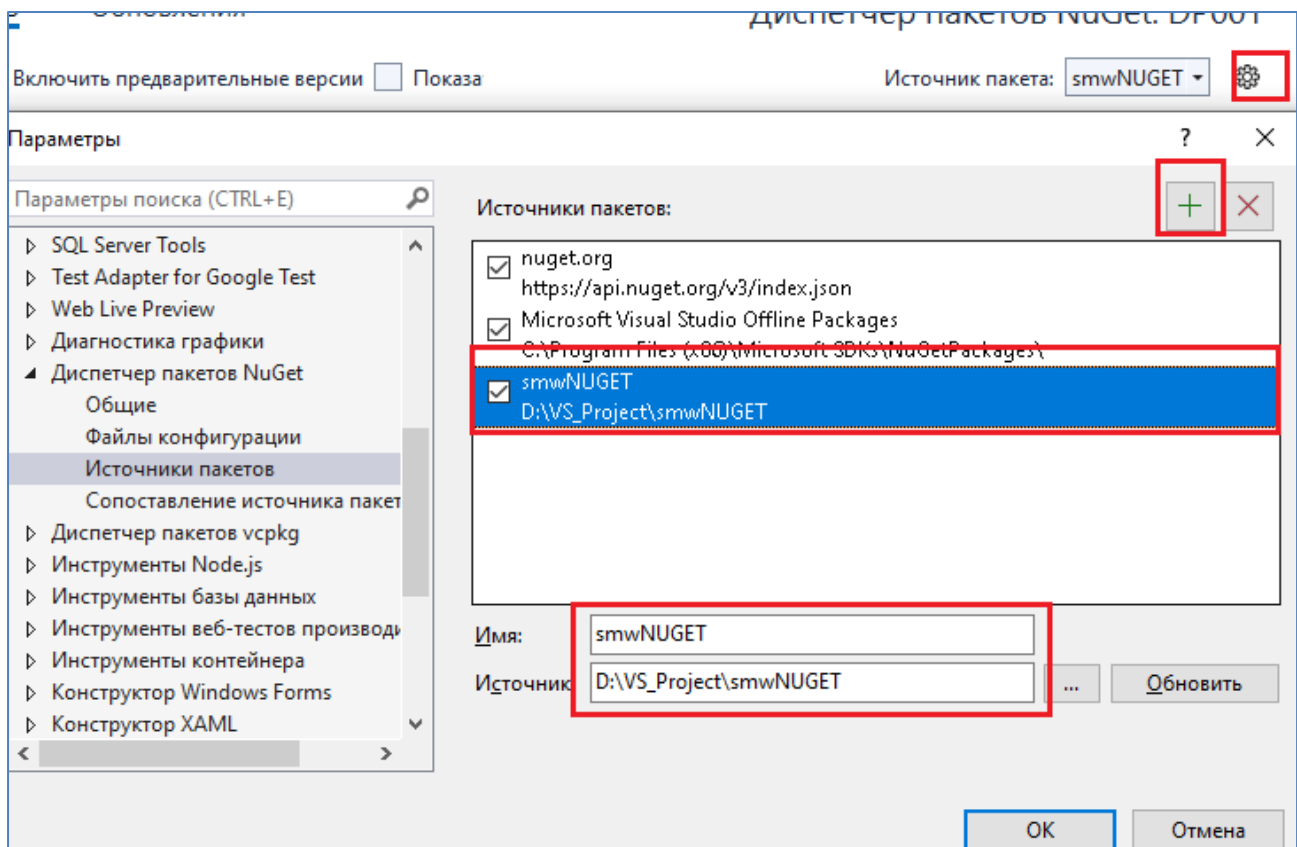
9. **NUGET**: создание пакета (терминал)

```
PowerShell для разработчиков
+ PowerShell для разработчиков
*****
** Visual Studio 2022 Developer PowerShell v17.11.3
** Copyright (c) 2022 Microsoft Corporation
*****
PS D:\VS_Project\DesignPatterns> dotnet pack --configuration Release
Определение проектов для восстановления...
Все проекты обновлены для восстановления.
DP001_lib -> D:\VS_Project\DesignPatterns\DP001_lib\bin\Release\net8.0\DP001_lib.dll
В пакете DP001_lib.1.0.0 отсутствует файл сведений. Перейдите на страницу https://aka.ms/nuget/authoring-best-practices/readme, чтобы узнать, почему файлы сведения о пакете важны.
Пакет "D:\VS_Project\DesignPatterns\DP001_lib\bin\Release\DP001_lib.1.0.0.nupkg" успешно создан.
DP001 -> D:\VS_Project\DesignPatterns\DP001\bin\Release\net8.0\DP001.dll
В пакете DP001.1.0.0 отсутствует файл сведений. Перейдите на страницу https://aka.ms/nuget/authoring-best-practices/readme, чтобы узнать, почему файлы сведения о пакете важны.
Пакет "D:\VS_Project\DesignPatterns\DP001\bin\Release\DP001.1.0.0.nupkg" успешно создан.
```

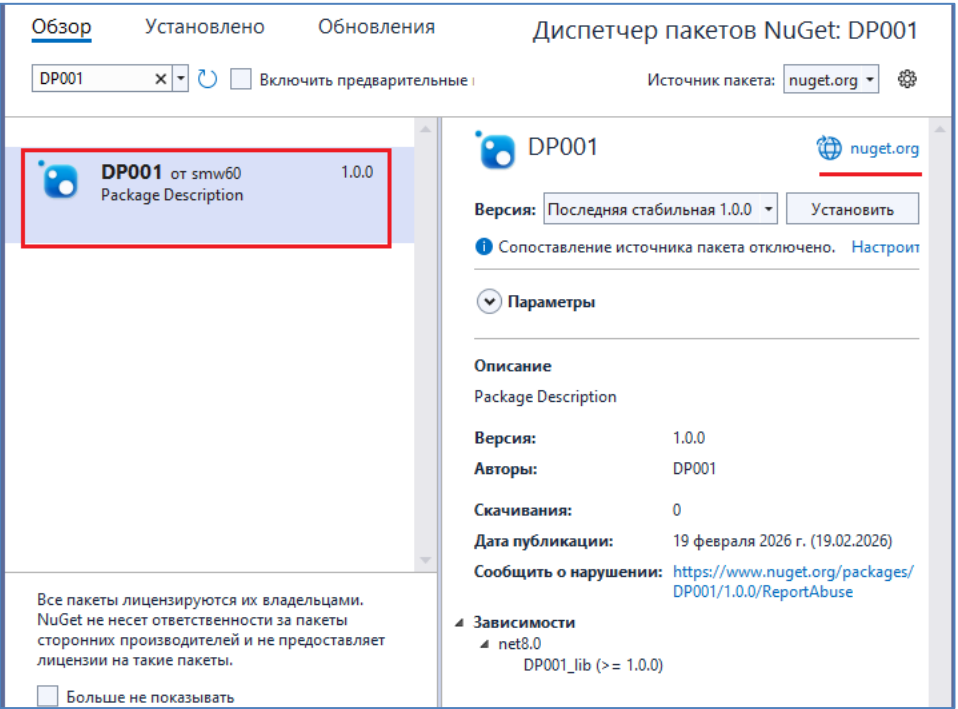
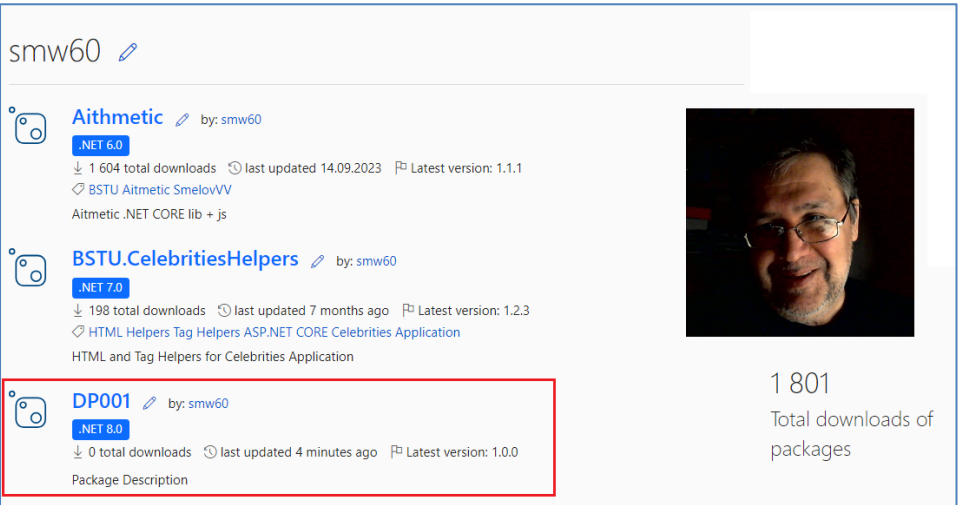
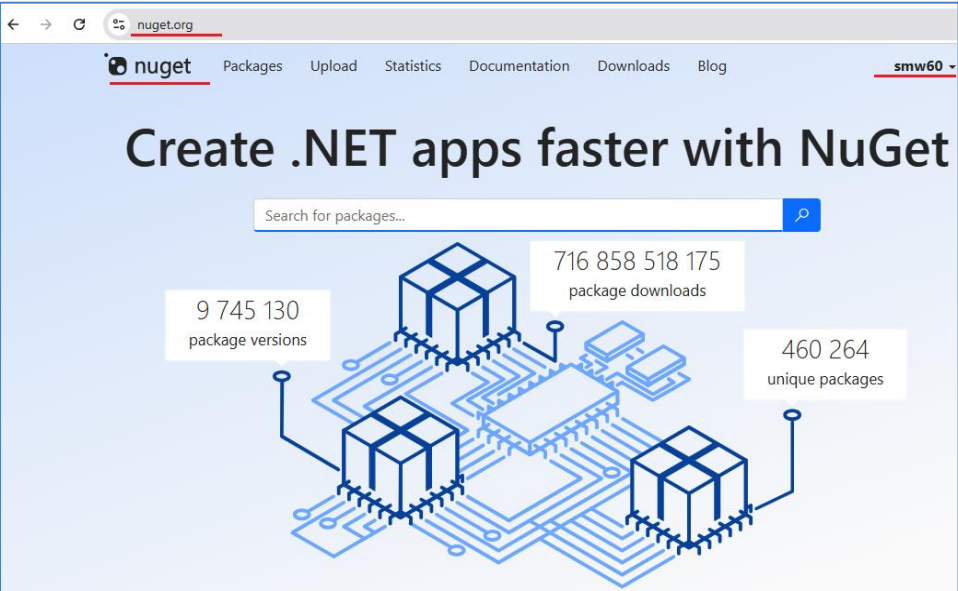
10. **NUGET**: создание пакета локального NUGET-репозитория

Открыть элемент как корень обозревателя
Новое представление: Обозреватель решений
Вложение файлов
Изменить файл проекта
Зависимости сборки
Добавить
Управление пакетами NuGet...
Управление секретами пользователей
Синхронизация пространства имен

```
D:\VS_Project>tree /f smwNUGET
Структура папок тома Новый том
Серийный номер тома: 000000CB BEA9:B183
D:\VS_PROJECT\SMWNUGET
  DP001_lib.1.0.0.nupkg
Подпапки отсутствуют
```



11. Применение репозитория **nuget.org**



12. **Конец.**